```
import numpy as np
students = np.array([
   [85,90,78], #student1
   [60,55,70],
   [95,92,88],
   [40,50,45],
   [72,68,75]
])
students
```

```
array([[85, 90, 78],
       [60, 55, 70],
       [95, 92, 88],
       [40, 50, 45],
       [72, 68, 75]])
```

```
avg_marks = np.mean(students,axis=1)
print("Average Marks:",avg_marks)
```

```
Average Marks: [84.33333333 61.66666667 91.66666667 45.         71.66666667]
```

```
topper = np.argmax(avg_marks)
print("Topper Student: ",topper)
```

```
Topper Student:  2
```

```
failed_student = np.where(avg_marks<50)
print("Failed student : ",failed_student[0]+1)
```

```
Failed student :  [4]
```

```
#performance Report
for i ,avg in enumerate(avg_marks):
  status="pass" if avg>50 else "Fail"
  print(f"Student {i+1}: Avg = {avg:.2f} | {status}")
```

```
Student 1: Avg = 84.33 | pass
Student 2: Avg = 61.67 | pass
Student 3: Avg = 91.67 | pass
Student 4: Avg = 45.00 | Fail
Student 5: Avg = 71.67 | pass
```

pandas

```
import pandas as pd
```

```
#create data frame and series
data={
    'Name': ['Prashant','Ashish','Sandip'],
    'Age':[25,30,35],
    'City':['New york','Paris','London']
}
df=pd.DataFrame(data)
print(df)
```

```
       Name  Age      City
0  Prashant   25  New york
1    Ashish   30     Paris
2    Sandip   35    London
```

```
df.shape
```

```
(3, 3)
```

```
df.columns
```

```
Index(['Name', 'Age', 'City'], dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
```

```
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
```

```
df.describe()
```

```
df['Age'].unique()
```

```
array([25, 30, 35])
```

```
#Data selection and filtering
df[['Name','Age']]
```

```
#row selection
df.iloc[0:, :-2] #[row:row,col:col]
```

```
#row selection
df.iloc[0:2, :-2] #[row:row,col:col]
```

```
#row selection
df.iloc[1:, :-2] #[row:row,col:col]
```

```
#row selection
df.iloc[1:2, :-2] #[row:row,col:col]
```

```
#row selection
df.iloc[1:2, :2] #[row:row,col:col]
```

```
#row selection by condition
df[df['Age']<=25]
```

```
#row selection by condition
df[df['Age']>=25]
```

```
#row selection by condition
df[df['Age']<=67]
```

```
#add column
df['Salary'] = [50000,100000,150000]
df
```

```
#add column
df['Company'] =['Apple','Google','Vipro']
df
```

```
#update a column
df['Age'] = df['Age']+1
df
```

```
#add column
df['Salary'] = df['Salary']+100000
df
```

```
#delete column
df.drop('Age',axis=1,inplace=True)
df
```

```
#add the rows
df.loc[3] = ['Anshul','Mumbai','NaNs']
df
```

```
#update a column
#df['Age'] = df['Age']+1
#df
```

```
#Grouping and aggregation
#df.groupby('City').agg({'Company': ['mean','max']})
```

```
#Sorting data
#df.sort_values(by='Age',ascending=True)
```

```
#value count
df['City'].value_counts()
pd.crosstab(df['City'],df['Company'])
```

```
import matplotlib.pyplot as plt
```

```
x=[1,2,3,4]
y=[2,4,6,8]
plt.plot(x,y)
```

```
x=[1,2,3,4]
y=[2,4,6,8]
plt.plot(x,y)
plt.title("Line Plot")
```

```
x=[1,2,3,4]
y=[2,4,6,8]
plt.plot(x,y)
plt.title("Line Plot")
plt.xlabel('X axis')
```

```
x=[1,2,3,4]
y=[2,4,6,8]
plt.plot(x,y)
plt.title("Line Plot")
plt.xlabel('X axis')
plt.ylabel('Y axis')
```

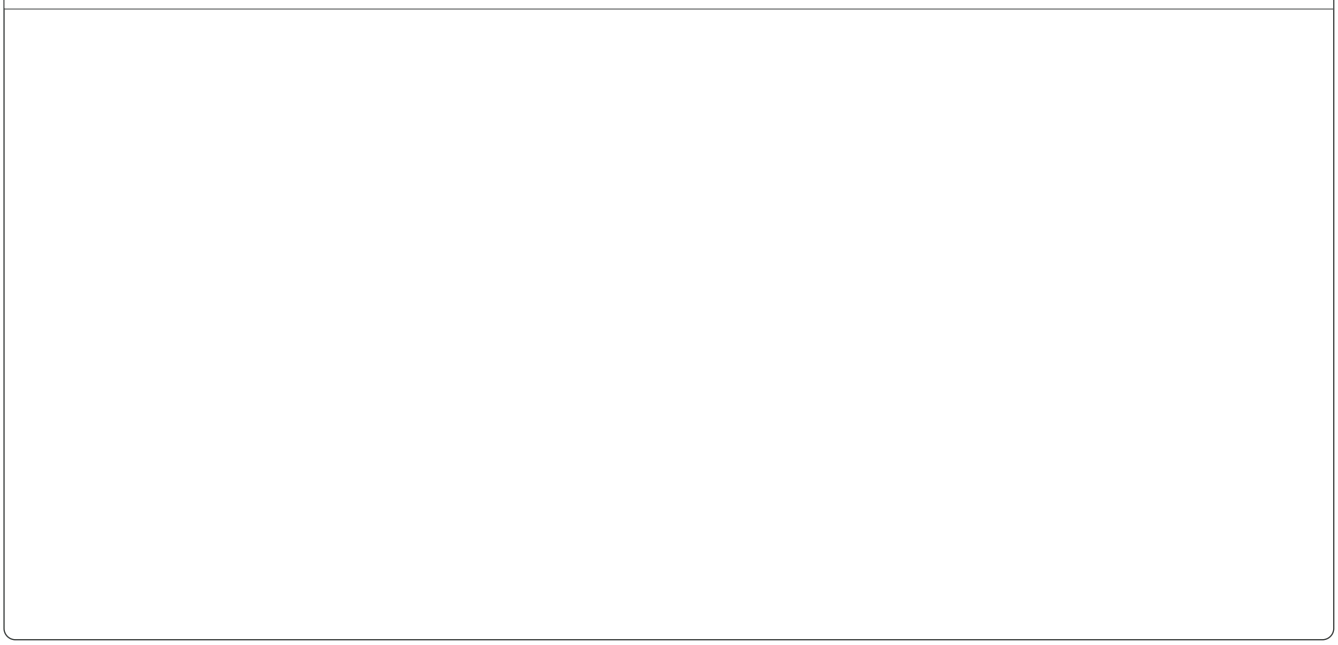
```
x=[1,2,3,4]
y=[2,4,6,8]
plt.plot(x,y)
plt.title("Line Plot")
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.grid(True)
```

```
x=[1,2,3,4]
y=[2,4,6,8]
plt.plot(x,y)
plt.title("Line Plot")
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.grid(True)
plt.show()
```

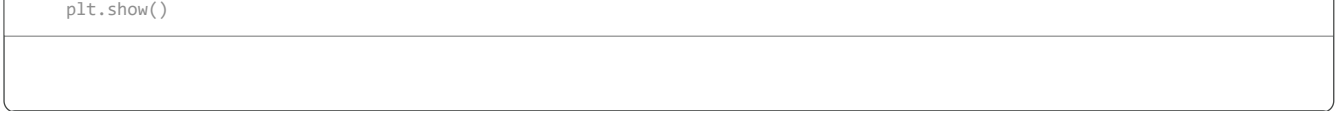
```
x=[1,2,3,4]
y=[2,4,6,8]
plt.plot(x,y)
plt.title("Line Plot")
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.grid(False)
```



```
x=[19,25,36,78]
y=[22,43,67,89]
plt.plot(x,y)
plt.title("Line Plot")
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.grid(True)
```



```
#Bar Chart
months = ['Jan','Feb','Mar','April']
sales = [250,300,400,350]
plt.bar(months,sales,color='orange')
plt.title('Monthly sales')
plt.xlabel('Months')
plt.ylabel('sales')
plt.show()
```



```
#Bar Chart
months = ['Jan','Feb','Mar','April']
sales = [250,300,400,350]
plt.barh(months,sales,color='orange')
plt.title('Monthly sales (Horizontal)')
plt.xlabel('Months')
plt.ylabel('sales')
plt.show()
```

```
#Bar Chart
months = ['Jan','Feb','Mar','April']
sales = [250,300,400,350]
plt.bar(months,sales,color='orange')
plt.title('Monthly sales')
plt.xlabel('Months')
plt.ylabel('sales')
plt.grid(True)
plt.show()
```

```
#Bar Chart
months = ['Jan','Feb','Mar','April']
sales = [250,300,400,350]
plt.barh(months,sales,color='orange')
plt.title('Monthly sales (Horizontal)')
plt.xlabel('Months')
plt.ylabel('sales')
plt.grid(True)
plt.show()
```

```
#Scatter plot(comparing Height vs Weight)
height = [150,160,170,180]
weight=[50,60,65,75]
plt.scatter(height,weight,color='green')
plt.title('Height vs Weight')
plt.xlabel('Height (cm)')
plt.ylabel('Weight(kg)')
plt.grid(True)
plt.show()
```

```
#pie chart(market share)
brands = ['Brand A','Brand B','Brand c']
shares = [45,30,25]
plt.pie(shares,labels=brands,autopct='%1.1f%%',startangle=58)
plt.title('Market Shares')
plt.axis('equal')
plt.show()
```

```
#Multiple plot in onje figure
x=[1,2,3,4]
y1=[1,4,9,16]
y2=[1,2,3,4]
plt.plot(x,y1,label='Square')
plt.plot(x,y2,label='Linear')
plt.legend()
plt.title('Multiple Lines')
plt.show()
```

```
#Subplot
fig,axs = plt.subplots(2,2)

axs[0,0].plot(x,y1)
axs[0,0].set_title('Square')

axs[0,1].bar(months,sales)
axs[0,1].set_title('Sales')

axs[1,0].scatter(height,weight)
axs[1,0].set_title('Scatter')

axs[1,1].pie(shares,labels=brands,autopct='%1.1f%%')
axs[1,1].set_title('Pie')
```

```
plt.tight_layout()
plt.show()
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
data = {
    'Name': ['A','B','C','D','E'],
    'Math' : [78,32,56,89,12],
    'Science':[78,43,32,90,23],
    'English': [43,65,35,87,33]
    }
df = pd.DataFrame(data)
print(df)
```

```
  Name  Math  Science  English
0    A    78       78       43
1    B    32       43       65
2    C    56       32       35
3    D    89       90       87
4    E    12       23       33
```

```
marks=df[["Math","Science","English"]].values
average_marks = np.mean(marks,axis=1)
df["Average"]=average_marks
print(df)
```

```
  Name  Math  Science  English    Average
0    A    78       78       43  66.333333
1    B    32       43       65  46.666667
2    C    56       32       35  41.000000
3    D    89       90       87  88.666667
4    E    12       23       33  22.666667
```

```
def grade(avg):
  if avg>= 90:
    return 'A'
  elif avg>= 75:
    return 'B'
  elif avg >=  60:
    return 'C'
  else:
    return 'D'
df['Grade']=df["Average"].apply(grade)
df
```

```
plt.figure()
plt.bar(df["Name"],df["Average"])
plt.xlabel('Students')
plt.ylabel('Average Marks')
plt.title('Average Marks of Students')
plt.show()
```

```
#average marks per subject
df[["Math","Science","English"]].mean().plot(kind="bar")
plt.title("Average Marks per Subject")
plt.ylabel("Marks")
plt.show()
```

Start coding or generate with AI.